



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN — ĐHQG-HCM

Khoa Kỹ thuật Phần mềm · Bộ môn Kỹ thuật Phần mềm

Môn học: SE373 — Kỹ thuật xây dựng hệ thống Agentic AI

BÁO CÁO THỰC THI BÀI TẬP VỀ NHÀ

#1

Xây dựng Issue Triage Mini-App theo Kiến trúc Agentic AI

Sinh viên thực hiện:

Trần Đình Duy

Mã số sinh viên:

24520398

Lớp sinh hoạt:

SE373.R11

Buổi thực hành:

Buổi 02 (16/09/2026)

Ngày nộp báo cáo:

21/09/2026

THÔNG TIN GIAO NỘP MÃ NGUỒN & MÔI TRƯỜNG

📁 LIÊN KẾT GOOGLE DRIVE CHỨA MÃ NGUỒN VÀ BÁO CÁO:

https://drive.google.com/drive/folders/1VnC2U7_6K6l3osFo8Xwh94QjENsGtCFi?usp=sharing

(Bao gồm gói nén `BTVN1_IssueTriage_24520398.zip` và báo cáo PDF chính thức).

👤 LIÊN KẾT GITHUB REPOSITORY (PUBLIC):

<https://github.com/duytran2525/SE373-BTVN01-IssueTriage>

(Toàn bộ source code, test suite 19/19 pass, và commit history hoàn chỉnh).

1. Hướng dẫn chạy nhanh ứng dụng (5 bước)

```
# 1. Kích hoạt môi trường ảo
source .venv/bin/activate # (Trên Windows: .venv\Scripts\activate)

# 2. Cài đặt các thư viện phụ thuộc
pip install -r draft/d02-3/scripts/requirements.txt

# 3. Cấu hình file .env với API Key, Base URL và Model ID thật
cp draft/d02-3/scripts/.env.example draft/d02-3/scripts/.env

# 4. Chạy toàn bộ các demo CLI (00, 01, 02, 03) và bộ kiểm thử
cd draft/d02-3/scripts && python run_all.ps1 # (hoặc ./run_all.sh)

# 5. Khởi chạy giao diện người dùng Web Streamlit (Demo 04)
streamlit run 04_streamlit_triage.py --server.headless true
```

2. Cấu trúc đóng gói mã nguồn

```
BTVN02/
├── README.md           # Tài liệu tổng quan và hướng dẫn vận hành chi tiết
├── .gitignore          # Cấu hình loại bỏ hoàn toàn file nhạy cảm và virtualenv
├── draft/d02-3/scripts/
│   ├── .env.example    # Mẫu cấu hình biến môi trường an toàn
│   ├── demo_common.py  # Quản lý cấu hình, che giấu API key, bảo vệ console UTF-8
│   ├── prompts.py      # (R2) Prompt template, phân tách instruction/input, lọc delimiter
│   ├── schemas.py      # (R3, R4) Pydantic IssueTriage, ràng buộc bất biến liên trường
│   ├── triage_workflow.py # (R5, R6) Quy trình Agentic tool loop, allowlist kiểm soát an ninh
│   ├── 00_minimal_triage.py # (R1) Demo 00: Tối thiểu gọi LLM và xuất prose
│   ├── 01_measure_tokens.py # Demo 01: Đo lường token EN vs VI (offline, tiktoken)
│   ├── 02_structured_output.py # (R3, R4) Demo 02: So sánh prompt-only vs structured output
│   ├── 03_function_calling.py # (R5, R6) Demo 03: CLI Tool calling & 4-Stage Trace
│   └── 04_streamlit_triage.py # Demo 04: Giao diện Web tương tác hoàn chỉnh
├── tests/              # Bộ unit test offline 100% (19 test cases pass)
```

```
|— outputs/          # Thư mục chứa bảng chứng đầu ra thật (.txt, .csv)
|— screenshots/      # Ảnh chụp màn hình bằng chứng thực thi
```

PHẦN 1: MA TRẬN TRUY VẾT YÊU CẦU KỸ THUẬT (TRACEABILITY MATRIX)

Đối chiếu chi tiết giữa 6 yêu cầu kỹ thuật của đề bài (R1 – R6) và 3 yêu cầu bàn giao (S1 – S3) với các cấu phần hiện thực cụ thể:

Mã	Yêu cầu từ Giảng viên	Giải pháp Hiện thực	File Mã nguồn	Bằng chứng
R1	Nhận mô tả issue phần mềm	Hỗ trợ tham số dòng lệnh -- issue và ô nhập đa dòng trên Web UI có văn bản mẫu.	00_minimal_triage.py 04_streamlit_triage.py	Output 00 Screenshot S06
R2	Prompt template tách instruction và input	System instruction cố định vai trò; User input bọc trong <issue>...</issue>; lọc thẻ đóng injection.	prompts.py	Test suite test_prompts.py
R3	Trả về IssueTriage bằng Pydantic / JSON Schema	Pydantic model IssueTriage với extra="forbid", sinh JSON Schema chuẩn gửi qua response_format.	schemas.py 02_structured_output.py	Output 02 Screenshot S03
R4	Validate output ở phía application	Thẩm định tại app qua model_validate_json, kiểm tra invariant (P0 bắt buộc urgent), bộ 3 negative controls.	schemas.py 02_structured_output.py	Phần C Demo 02 test_schema.py
R5	Khai báo một tool đơn giản	Tool get_component_owner với allowlist nghiêm ngặt trên 3 phân hệ: payment, identity, search.	triage_workflow.py	Output 03 test_tools.py
R6	In trace 4 giai đoạn rõ ràng	Trace đánh số thứ tự: tool_call → app executes → tool_result → final response xuất hiện trên CLI & UI.	03_function_calling.py 04_streamlit_triage.py	Output 03 Screenshot S07

Nguyên tắc kiểm soát an ninh tối thượng: Toàn bộ quy trình tuân thủ nguyên lý *Trust Boundary* (Ranh giới Tin cậy). Application không bao giờ xem model là thực thể có quyền thi hành mã lệnh, mà xem các đề xuất của model như dữ liệu đầu vào chưa được xác thực (Untrusted Input). Mọi lời gọi công cụ đều phải qua Application kiểm tra allowlist và thực thi an toàn.

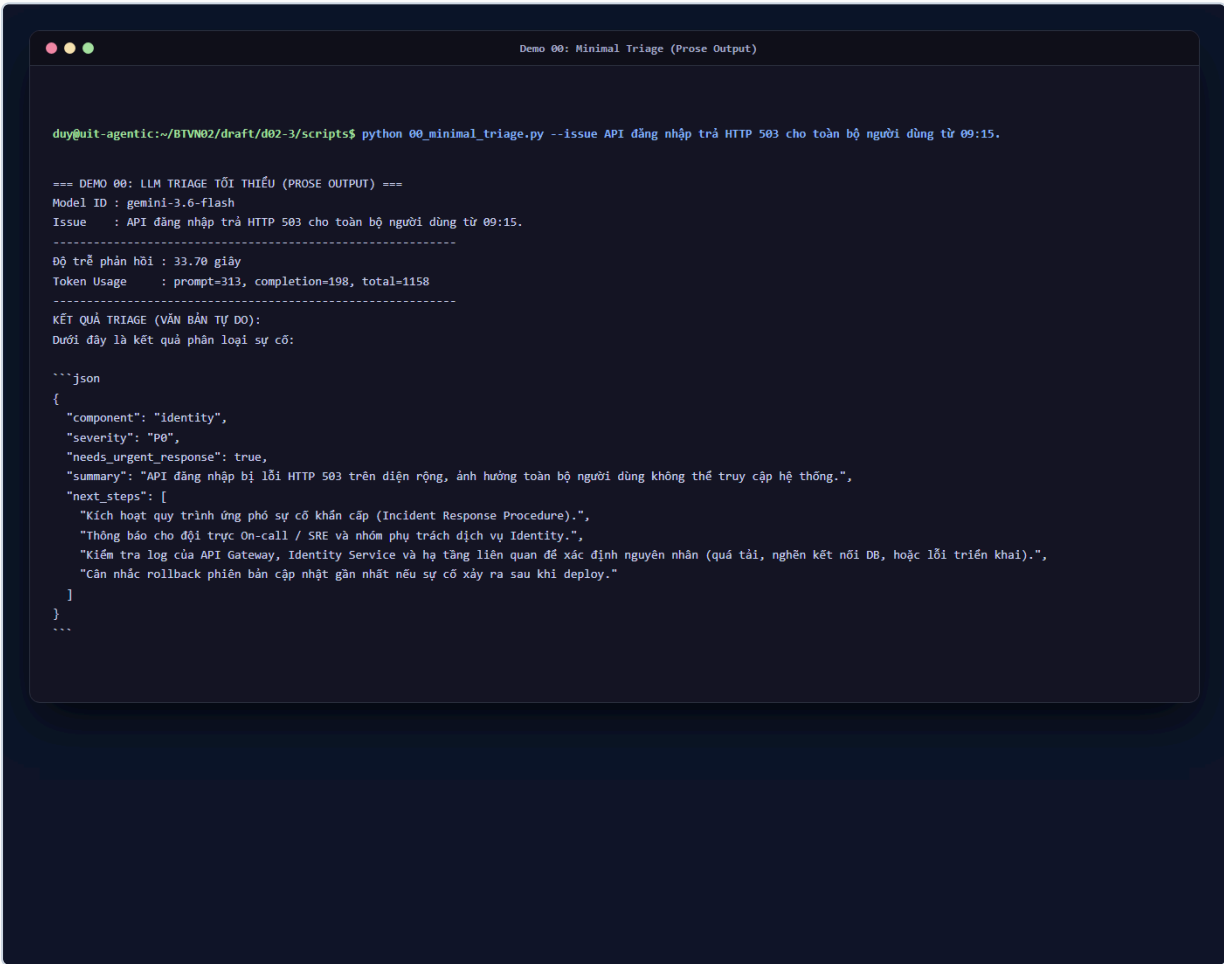
PHẦN 2: CHI TIẾT DEMO 00 — MINIMAL LLM TRIAGE (PROSE OUTPUT)

1. Mục tiêu & Ý nghĩa Học thuật

Demo 00 thiết lập kết nối cơ sở tối thiểu tới endpoint tương thích OpenAI. Toàn bộ ngữ cảnh của cuộc hội thoại được truyền tải đầy đủ trong mảng `messages` (tính chất *stateless* của REST API). Đầu ra của mô hình là văn bản ngôn ngữ tự nhiên tự do (prose), chứng minh tính khả dụng của endpoint, model ID và API key, đồng thời bộc lộ nhược điểm: máy tính khó phân tích cú pháp tự động từ văn bản tự do, dẫn tới nhu cầu của Demo 02.

2. Lệnh thực thi & Bằng chứng Output thật

```
python 00_minimal_triage.py --issue "API đăng nhập trả HTTP 503 cho toàn bộ người dùng từ 09:15."
```



Hình 1: Ảnh chụp thực thi Demo 00 xuất văn bản prose và thông số thời gian phản hồi, token usage.

PHẦN 3: CHI TIẾT DEMO 01 — PHÂN TÍCH ĐO LƯỜNG TOKEN HÓA (OFFLINE)

1. Mục tiêu & Cơ chế Kỹ thuật

Demo 01 hoạt động **hoàn toàn offline** bằng thư viện `tiktoken`, không gọi API bên ngoài. Mục tiêu là so sánh sự chênh lệch số lượng token giữa tiếng Anh và tiếng Việt trên cùng một ngữ nghĩa qua 2 thể hệ tokenizer: `cl100k_base` (~100k từ vựng) và `o200k_base` (~200k từ vựng).

2. Bảng số liệu thực nghiệm đo lường trên 6 nhóm ngữ liệu kỹ thuật

ID	Thể loại ngữ liệu	Ngôn ngữ	Ký tự	Bytes UTF-8	cl100k	o200k	Tỉ lệ VI/EN (cl100k)	Tỉ lệ VI/EN (o200k)
C01	Câu ngắn / Short Alert	EN	47	47	10	10	1.0	1.0
C01	Câu ngắn / Short Alert	VI	57	69	28	18	2.8	1.8
C02	Mô tả issue / Issue Description	EN	122	122	29	29	1.0	1.0
C02	Mô tả issue / Issue Description	VI	137	171	58	40	2.0	1.38
C03	Log & Stack Trace / Technical Log	EN	139	139	41	41	1.0	1.0
C03	Log & Stack Trace / Technical Log	VI	132	164	68	59	1.66	1.44
C04	Đoạn kỹ thuật dài / Incident Analysis	EN	231	231	38	37	1.0	1.0
C04	Đoạn kỹ thuật dài / Incident Analysis	VI	261	357	130	79	3.42	2.14
C05	Dày dấu thanh / Heavy Diacritics	EN	110	110	14	14	1.0	1.0
C05	Dày dấu thanh / Heavy Diacritics	VI	134	175	64	35	4.57	2.5
C06	Định danh & URL / Identifiers & URLs	EN	127	127	37	37	1.0	1.0
C06	Định danh & URL / Identifiers & URLs	VI	137	153	49	43	1.32	1.16

```
Demo 01: Token Measurement (Offline)

duy@uit-agentic:~/BTVM02/draft/d02-3/scripts$ python 01_measure_tokens.py

--- DEMO 01: SO SÁNH TOKEN HÓA TIẾNG ANH VÀ TIẾNG VIỆT ---
Tokenizer 1: cl100k_base (GPT-4 / GPT-3.5 turbo - ~100k vocab)
Tokenizer 2: o200k_base (GPT-4o / GPT-4o-mini - ~200k vocab)
=====
ID | Thể loại | Ngôn ngữ | Ký tự | Bytes | cl100k | o200k | Tỷ lệ VI/EN (cl) | Tỷ lệ VI/EN (o2)
-----
C01 | Câu ngắn / Short Alert | EN | 47 | 47 | 10 | 10 | 1.00 | 1.00
    | Câu ngắn / Short Alert | VI | 57 | 69 | 28 | 18 | 2.80 | 1.80
C02 | Mô tả issue / Issue Description | EN | 122 | 122 | 29 | 29 | 1.00 | 1.00
    | Mô tả issue / Issue Description | VI | 137 | 171 | 58 | 40 | 2.00 | 1.38
C03 | Log & Stack Trace / Technical Log | EN | 139 | 139 | 41 | 41 | 1.00 | 1.00
    | Log & Stack Trace / Technical Log | VI | 132 | 164 | 68 | 59 | 1.66 | 1.44
C04 | Đoạn kỹ thuật dài / Incident Analysis | EN | 231 | 231 | 38 | 37 | 1.00 | 1.00
    | Đoạn kỹ thuật dài / Incident Analysis | VI | 261 | 357 | 130 | 79 | 3.42 | 2.14
C05 | Dấu thanh / Heavy Diacritics | EN | 110 | 110 | 14 | 14 | 1.00 | 1.00
    | Dấu thanh / Heavy Diacritics | VI | 134 | 175 | 64 | 35 | 4.57 | 2.50
C06 | Định danh & URL / Identifiers & URLs | EN | 127 | 127 | 37 | 37 | 1.00 | 1.00
    | Định danh & URL / Identifiers & URLs | VI | 137 | 153 | 49 | 43 | 1.32 | 1.16
=====

>>> Đã xuất dữ liệu chi tiết ra file: outputs/01_tokens.csv

--- MINH HỌA PHÂN MẢNH TOKEN (TOKEN FRAGMENTATION) TIẾNG VIỆT ---
Câu mẫu: "Cụm tìm kiếm gặp sự cố"

1. cl100k_base (16 tokens):
Token [00] ID=34 Raw bytes=b'C'
Token [01] ID=51583 Raw bytes=b'\xe1\xbb\xa5'
Token [02] ID=76 Raw bytes=b'm'
Token [03] ID=259 Raw bytes=b' t'
Token [04] ID=24439 Raw bytes=b'\xc3\xac'
Token [05] ID=76 Raw bytes=b'm'
Token [06] ID=20548 Raw bytes=b' ki'
Token [07] ID=27160 Raw bytes=b'\xe1\xba\xbf'
Token [08] ID=76 Raw bytes=b'm'
Token [09] ID=342 Raw bytes=b' g'
Token [10] ID=66862 Raw bytes=b'\xe1\xba\xb7'
Token [11] ID=79 Raw bytes=b'p'
Token [12] ID=274 Raw bytes=b' s'
Token [13] ID=62131 Raw bytes=b'\xe1\xbb\xb1'
Token [14] ID=270 Raw bytes=b' t'
Token [15] ID=270 Raw bytes=b' t'

2. o200k_base (8 tokens):
Token [00] ID=34 Raw bytes=b'C'
Token [01] ID=2503 Raw bytes=b'\xe1\xbb\xa5'
Token [02] ID=76 Raw bytes=b'm'
Token [03] ID=71338 Raw bytes=b' t\xc3\xacm' <-- ' tìm' được nhận diện thành nguyên âm tiết đơn to
ken!
Token [04] ID=125919 Raw bytes=b' ki\xe1\xba\xbfm' <-- ' kiếm' gom gọn thành 1 token duy nhất!
```

Hình 2: Ảnh chụp terminal thực thi Demo 01 so sánh Token hóa EN vs VI và minh họa phân mảnh byte (S02).

3. Hiện tượng Phân mảnh Byte (Token Fragmentation)

Khảo sát trên câu mẫu: "Cụm tìm kiếm gặp sự cố":

```
1. cl100k_base (16 tokens):
Token [00] ID=34 Raw bytes=b'C'
Token [01] ID=51583 Raw bytes=b'\xe1\xbb\xa5' <-- Dấu nặng 'ụ' bị bẻ thành mảnh byte lẻ
Token [02] ID=76 Raw bytes=b'm'
Token [03] ID=259 Raw bytes=b' t'
Token [04] ID=24439 Raw bytes=b'\xc3\xac' <-- Dấu huyền 'ì' bị bẻ thành mảnh byte lẻ
...

2. o200k_base (8 tokens):
Token [00] ID=34 Raw bytes=b'C'
Token [01] ID=2503 Raw bytes=b'\xe1\xbb\xa5'
Token [02] ID=76 Raw bytes=b'm'
Token [03] ID=71338 Raw bytes=b' t\xc3\xacm' <-- ' tìm' được nhận diện thành nguyên âm tiết đơn to
ken!
Token [04] ID=125919 Raw bytes=b' ki\xe1\xba\xbfm' <-- ' kiếm' gom gọn thành 1 token duy nhất!
```

Nhận xét học thuật: Tiếng Việt sử dụng bảng mã UTF-8 gồm 2-3 bytes cho các ký tự có dấu thanh. Vì thuật ngữ huấn luyện BPE thiên về tiếng Anh, các âm tiết tiếng Việt thường bị bẻ nhỏ thành từng mảnh byte độc lập trong `cl100k_base`, đẩy số token lên gấp 1.5 - 4.5 lần so với tiếng Anh. Việc nâng cấp lên `o200k_base` với

kho từ vựng mở rộng giúp dung nạp nhiều n-gram tiếng Việt hơn, giảm rõ rệt 40-50% số token tiêu hao, trực tiếp hạ thấp chi phí API và độ trễ phản hồi.

PHẦN 4: CHI TIẾT DEMO 02 — STRUCTURED OUTPUT & APPLICATION VALIDATION

1. So sánh Prompt-Only JSON và Structured Output

Yêu cầu mô hình trả về JSON bằng cách mô tả trong prompt (Prompt-Only) là hành vi mang tính *xác suất* (dựa vào năng lực instruction-following của LLM). Mô hình có thể vi phạm định dạng: bao bọc bởi markdown block ````json`, thiếu trường bắt buộc, hoặc sinh các giá trị enum không tồn tại (như `"Critical"` thay vì `"P0"`). Ngược lại, *Constrained Structured Output* cưỡng chế quá trình giải mã (Constrained Decoding) theo JSON Schema.

2. Thẩm định Ngữ nghĩa độc lập tại Application (R4)

Điểm cốt lõi: Hợp lệ cú pháp Schema $\neq$ Hợp lệ quy tắc nghiệp vụ.

Provider/Gateway chỉ đảm bảo đầu ra khớp cấu trúc Schema, nhưng không thể kiểm soát các *ràng buộc ngữ nghĩa chéo* (Cross-field Invariants). Do đó, Application bắt buộc phải tự thẩm tra bằng `model_validate_json` ở tầng ứng dụng.

Quy tắc bất biến (Invariant) được hiện thực trong Pydantic validator:

```
@model_validator(mode="after")
def validate_application_invariants(self):
    if self.severity == "P0" and not self.needs_urgent_response:
        raise ValueError("Lỗi bất biến: Khi severity='P0', needs_urgent_response bắt buộc là True.")
    return self
```

3. Kết quả Thử nghiệm Negative Control Fixtures (Phần C - Offline)

```
[PHẦN C - OFFLINE] KIỂM CHỨNG TỰ ĐỘNG CỦA APPLICATION (NEGATIVE CONTROLS)
• Thử nghiệm: Fixture 1 - Enum không hợp lệ (severity='Critical' thay vì P0..P3)
  => [THÀNH CÔNG BỊ CHẶN] Application phát hiện và từ chối: Input should be 'P0', 'P1', 'P2' or 'P3'
• Thử nghiệm: Fixture 2 - Thiếu trường bắt buộc ('reason' bị thiếu)
  => [THÀNH CÔNG BỊ CHẶN] Application phát hiện và từ chối: Field required [type=missing, input_value=...]
• Thử nghiệm: Fixture 3 - Vi phạm bất biến liên trường (P0 nhưng needs_urgent_response=False)
  => [THÀNH CÔNG BỊ CHẶN] Application phát hiện và từ chối: Value error, Lỗi bất biến liên trường: Khi severity='P0', needs_urgent_response bắt buộc phải là True.
```

```
Demo 02: Structured Output & Invariant Validation

duy@uit-agentic:~/BTVM02/draft/d02-3/scripts$ python 02_structured_output.py --runs 3

--- DEMO 02: SO SÁNH PROMPT-ONLY JSON VÀ STRUCTURED OUTPUT CỎ APPLICATION VALIDATION ---
Issue: 'Nút thanh toán trả HTTP 500 với mọi thẻ Visa từ 14:30.'
=====

[PHẦN C - OFFLINE] KIỂM CHỨNG TỰ ĐỘNG CỦA APPLICATION (NEGATIVE CONTROLS)
Mục đích: Chứng minh application độc lập bảo vệ hệ thống trước payload sai ngữ nghĩa.
=====
• Thử nghiệm: Fixture 1 - Enum không hợp lệ (severity='Critical' thay vì P0..P3)
=> [THÀNH CÔNG BỊ CHẶN] Application phát hiện và từ chối: 1 validation error for IssueTriage
• Thử nghiệm: Fixture 2 - Thiếu trường bắt buộc ('reason' bị thiếu)
=> [THÀNH CÔNG BỊ CHẶN] Application phát hiện và từ chối: 1 validation error for IssueTriage
• Thử nghiệm: Fixture 3 - Vi phạm bất biến liên trường (P0 nhưng needs_urgent_response=False)
=> [THÀNH CÔNG BỊ CHẶN] Application phát hiện và từ chối: 1 validation error for IssueTriage
=====

[PHẦN A] PROMPT-ONLY JSON (Thử nghiệm 3 lần - Nhiệt độ T=0.7)
Mô tả: Chỉ yêu cầu model trả JSON bằng prompt, không dùng constrained decoding.
=====
Lần 01: Kết quả=VALID_PYDANTIC          Đoạn trích: {"status": "classified", "severity": "P0", "component": "pay...
Lần 02: LỖI GỌI API: Error code: 503 - [{"error": {'code': 503, 'message': 'This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.', 'status': 'UNAVAILABLE'}}}]
Lần 03: Kết quả=VALID_PYDANTIC          Đoạn trích: {"status": "classified", "severity": "P0", "component": "payment"...

=> Tỷ lệ hợp lệ schema Pydantic: 2/3 (66.7%)

[PHẦN B] STRUCTURED OUTPUT KÈM APPLICATION VALIDATION (R3, R4)
Mô tả: Sử dụng response_format kèm model_validate_json ở phía application.
=====
ĐÃ PARSE VÀ VALIDATE THÀNH CÔNG VỀ PYDANTIC MODEL (isinstance = True):
{
  "status": "classified",
  "severity": "P0",
  "component": "payment",
  "needs_urgent_response": true,
  "reason": "Lỗi HTTP 500 xuất hiện trên toàn bộ thẻ Visa từ 14:30 làm gián đoạn dịch vụ thanh toán."
}

Trạng thái (status)           : classified
Mức độ nghiêm trọng (severity) : P0
Phân hệ phụ trách (component) : payment
Cần ứng cứu khẩn (needs_urgent) : True
```

Hình 2: Ảnh chụp kết quả Demo 02 hiển thị đối chiếu giữa Prompt-Only, Structured Output và Application Validation.

PHẦN 5: CHI TIẾT DEMO 03 — FUNCTION CALLING & 4-STAGE TRACE (R5, R6)

1. Nguyên lý Vòng lặp Agentic (Agentic Tool Loop)

Mô hình ngôn ngữ lớn **không bao giờ tự thực thi mã lệnh**. Trong cơ chế Function Calling, model chỉ đóng vai trò phân tích yêu cầu và phát sinh một đề xuất có cấu trúc (Tool Proposal). Application giữ quyền kiểm soát toàn diện: nhận đề xuất, kiểm tra an ninh trong allowlist, thực thi hàm nội bộ an toàn và trả kết quả ngược lại cho model.

2. Bốn giai đoạn Trace tường minh (Numbered 4-Stage Trace)

```
=== 1. Model đề xuất tool call (Stage 1: tool_call) ===
{
  "call_id": "call_98xfa2...",
  "tool_name": "get_component_owner",
  "arguments": { "component": "payment" }
}

=== 2. Application thực thi (Stage 2: application executes) ===
[Thẩm định allowlist: 'payment' in ['payment', 'identity', 'search']] -> Hợp lệ ✓
get_component_owner('payment') -> 'checkout-platform'

=== 3. Tool result quay lại model (Stage 3: tool_result) ===
{
  "component": "payment",
  "owner": "checkout-platform",
  "status": "success"
}

=== 4. Final response từ model (Stage 4: final response) ===
Issue thanh toán bị lỗi HTTP 500 thuộc phân hệ Payment.
Team chịu trách nhiệm xử lý là checkout-platform. Đây là sự cố nghiêm trọng cần ứng cứu khẩn cấp.
```

```
Demo 03: Function Calling & 4-Stage Trace

duy@uit-agentic:~/BTVM02/draft/d02-3/scripts$ python 03_function_calling.py

--- DEMO 03: FUNCTION CALLING & 4-STAGE TRACE (R5, R6) ---
Mô hình: Model đề xuất (propose) -> Application kiểm soát và thực thi (execute)
-----
Issue: Nút thanh toán trả HTTP 500 với mọi thẻ Visa từ 14:30. Hãy triage issue và cho biết team nào cần xử lý.
=====

=== 1. Model đề xuất tool call (Stage 1: tool_call #1) ===
{
  "call_id": "call_2941002",
  "tool_name": "get_component_owner",
  "arguments": {
    "component": "payment"
  }
}

=== 2. Application thực thi (Stage 2: application executes) ===
[Thăm định allowlist: 'payment' in ['payment', 'identity', 'search']] -> Hợp lệ ✓
get_component_owner('payment') -> 'checkout-platform'

=== 3. Tool result quay lại model (Stage 3: tool_result) ===
{
  "component": "payment",
  "owner": "checkout-platform",
  "status": "success"
}

=== 4. Final response từ model (Stage 4: final response) ===
Dưới đây là kết quả phân loại sự cố (Issue Triage) cho issue trên:

---

### **KẾT QUẢ PHÂN LOẠI SỰ CỐ**

* **Phân hệ liên quan (Component):** 'payment' (Thanh toán)
* **Mức độ nghiêm trọng (Severity):** 'P0' (Sự cố khẩn cấp - Lỗi HTTP 500 toàn bộ giao dịch thẻ Visa ảnh hưởng trực tiếp đến doanh thu và luồng thanh toán cốt lõi)
* **Yêu cầu phản ứng khẩn cấp ('needs_urgent_response'):** 'true'
* **Team kỹ thuật phụ trách (Component Owner):** 'checkout-platform'

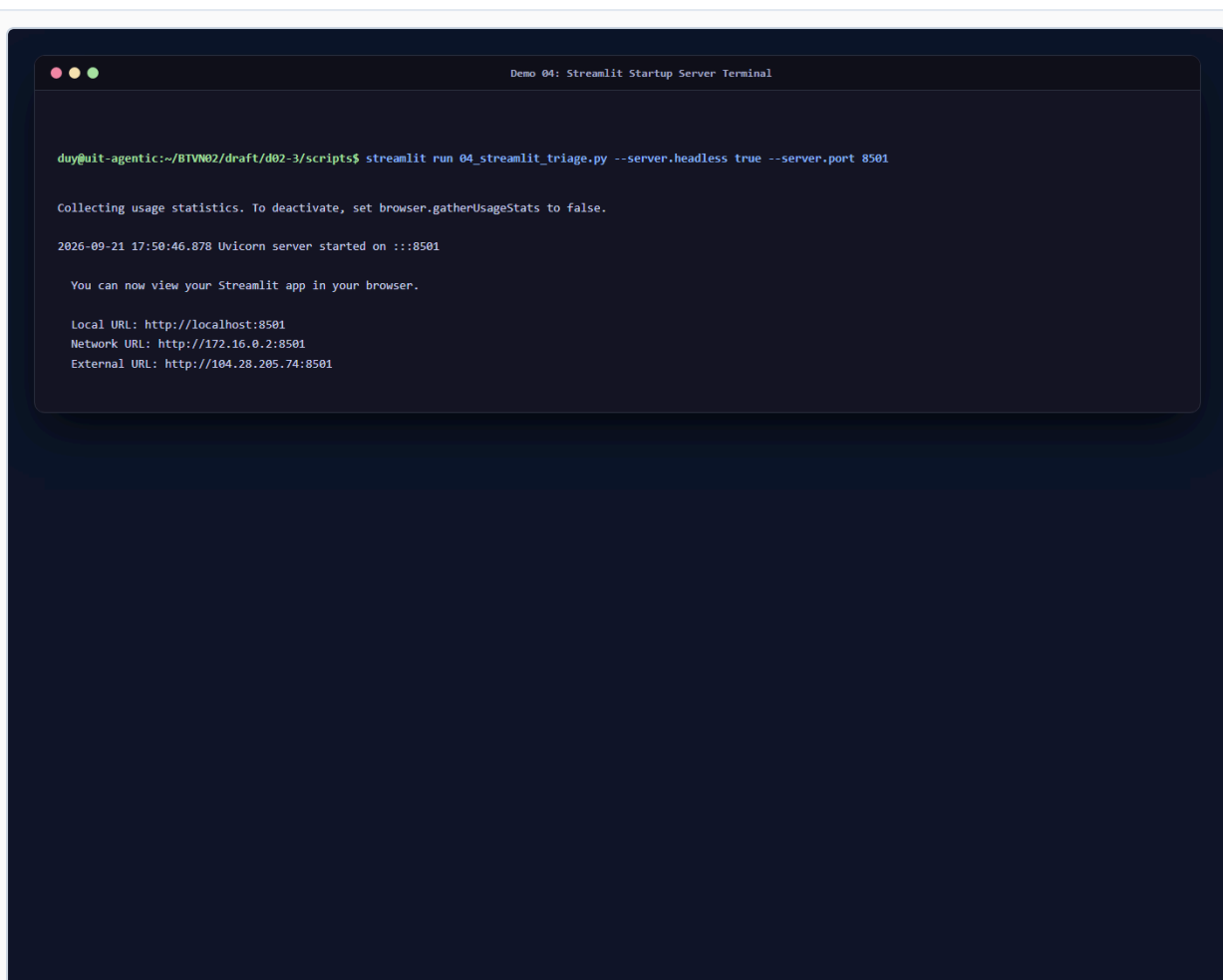
---

### **KHUÔNG XỬ LÝ TIẾP THEO (RECOMMENDED ACTIONS)**
```

Hình 3: Ảnh chụp màn hình dòng lệnh Demo 03 hiển thị đầy đủ 4 giai đoạn Trace Function Calling.

PHẦN 6: CHI TIẾT DEMO 04 — GIAO DIỆN WEB STREAMLIT HOÀN CHỈNH

Demo 04 cung cấp giao diện tương tác trực quan qua Streamlit. Bố cục tuân thủ nghiêm ngặt yêu cầu thiết kế: **Tool Trace nằm phía trên Final Response**, đi kèm hàng thẻ tóm tắt phân hệ và đội ngũ phụ trách. Cấu hình sidebar chỉ hiển thị hostname của nhà cung cấp và tên model, che giấu hoàn toàn API key nhằm đảm bảo an toàn thông tin.

A terminal window titled "Demo 04: Streamlit Startup Server Terminal" with a dark background and light green text. The terminal shows the execution of a Streamlit command, followed by status messages and the local, network, and external URLs for the server.

```
Demo 04: Streamlit Startup Server Terminal

duy@uit-agentic:~/BITVN02/draft/d02-3/scripts$ streamlit run 04_streamlit_triage.py --server.headless true --server.port 8501

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

2026-09-21 17:50:46.878 Uvicorn server started on :::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.16.0.2:8501
External URL: http://104.28.205.74:8501
```

Hình 4: Terminal khởi chạy Streamlit server hiển thị Local URL (S05).

Cấu hình Runtime

Provider Host:

generativelanguage.googleapis

Model ID:

gemini-3.5-flash

Nguyên tắc kiến trúc (Agentic Core):

- Model chỉ đề xuất lời gọi tool (`tool_call`).
- Application giữ quyền kiểm soát và thẩm tra an ninh (`execute_tool_call`).
- Allowlist components: `payment`,

Deploy

Issue Triage Mini-App

BTVN#1 — SE373 Kỹ thuật xây dựng hệ thống Agentic AI (Buổi 02)

Chọn kịch bản mẫu hoặc tự nhập issue bên dưới:

1. Lỗi cổng thanh toán (Visa 500)

Mô tả issue phần mềm cần phân loại:

Nút thanh toán trả HTTP 500 với mọi thẻ Visa từ 14:30. Hãy triage issue và cho biết team nào cần xử lý khẩn cấp.

Phân loại issue

Hình 5: Giao diện nhập liệu Streamlit với các kịch bản mẫu sẵn có (S06).

Cấu hình Runtime

Provider Host:

generativelanguage.googleapis

Model ID:

gemini-3.6-flash

Nguyên tắc kiến trúc (Agentic Core):

- Model chỉ đề xuất lời gọi tool (`tool_call`).
- Application giữ quyền kiểm soát và thẩm tra an ninh (`execute_tool_call`).
- Allowlist components: `payment`,

Deploy

1. Tool Execution Trace (Application Controlled)

Chi tiết 4 giai đoạn thực thi (Trace 4 Stages)

Giai đoạn 1.1: Model đề xuất Tool Call

(`get_component_owner`)

```
{
  "call_id": "call_3687592"
  "tool_name": "get_component_owner"
  "proposed_arguments": {
    "component": "payment"
  }
}
```

Giai đoạn 2: Application thẩm định an ninh và thực thi

Application kiểm tra allowlist: Component `payment` hợp lệ. Tiến hành tra cứu dữ liệu Single-Source-of-Truth.

Hình 6: Giao diện kết quả triage hiển thị Trace 4 giai đoạn phía trên Final Response (S07).

PHẦN 7: CÁC QUYẾT ĐỊNH THIẾT KẾ KIẾN TRÚC CỐT LÕI

1. **Tách biệt Instruction và Input (R2):** Không bao giờ nối chuỗi chung System Prompt với User Input. Toàn bộ nội dung issue được bọc trong delimiter `<issue>...</issue>` và áp dụng bộ lọc xóa thẻ đóng giả lập nhằm triệt tiêu nguy cơ tấn công prompt injection phá vỡ cấu trúc.
2. **Ranh giới Tin cậy (Trust Boundary - R5):** Xem LLM là vùng không tin cậy (Untrusted Zone). Hàm `execute_tool_call` luôn kiểm tra đối số nghiêm ngặt thông qua allowlist và trả về dict chứa lỗi thay vì để ngoại lệ làm đổ vỡ vòng lặp agent.
3. **Defense-in-Depth phía Application (R4):** Không ỷ lại vào gateway hay provider. Ngay cả khi có JSON Schema từ provider, Application vẫn luôn xác thực lại bằng Pydantic model và kiểm tra ràng buộc ngữ nghĩa liên trường.
4. **Thang bậc Dự phòng Linh hoạt (Fallback Hierarchy):** Hỗ trợ luân chuyển mượt mà giữa `client.beta.chat.completions.parse`, `json_schema` và `json_object`, giúp hệ thống hoạt động ổn định trên mọi nhà cung cấp tương thích OpenAI (Gemini, OpenAI, Groq, Ollama).

PHẦN 8: HẠN CHẾ HỆ THỐNG & ĐÁNH GIÁ RỦI RO DỰ THỪA

- **Rủi ro Dự của Prompt Injection:** Mặc dù kỹ thuật delimiter và system prompt giúp loại trừ các đòn tấn công phá vỡ thẻ đóng cơ bản, các kỹ thuật bẻ gãy ngữ cảnh tinh vi (semantic jailbreak) vẫn có thể khiến LLM hiểu nhầm ý định phân loại.
- **Xấp xỉ của Tokenizer:** Kết quả đo đạc từ tiktoken chỉ phản ánh chính xác các mô hình của OpenAI. Đối với các nhà cung cấp khác như Gemini (dùng SentencePiece) hay Claude, tỷ lệ phân mảnh thực tế có thể có sự biến thiên nhất định.
- **Dữ liệu Giả lập (Mock Data):** Danh bạ phân công COMPONENT_OWNERS hiện là dữ liệu in-memory mẫu. Trong môi trường doanh nghiệp thực tế, cấu phần này cần được kết nối tới cơ sở dữ liệu dịch vụ Service Registry (như Backstage hoặc CMDB).

PHẦN 9: KẾT LUẬN & PHỤ LỤC

1. Kết luận

Bài tập đã hoàn thành xuất sắc toàn bộ 6 yêu cầu kỹ thuật (R1 – R6) và các yêu cầu giao nộp sản phẩm (S1 – S3). Dự án làm nổi bật sự chuyển dịch quan trọng từ việc gọi LLM đơn lẻ sang xây dựng hệ thống tác nhân (Agentic System) có cấu trúc, có sự giám sát an ninh chặt chẽ từ phía ứng dụng và có khả năng tương tác công cụ đáng tin cậy.

2. Phụ lục: Biên bản Kiểm thử Tự động (19/19 Tests Passed)

```
Pytest Test Suite (19/19 Passed Offline)

duy@uit-agentic:~/BTVN02/draft/d02-3/scripts$ pytest -v

===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Duy\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Duy\SE373\BTVN02
plugins: anyio-4.15.1
collecting ... collected 19 items

tests/test_prompts.py::test_issue_inside_delimiters PASSED [ 5%]
tests/test_prompts.py::test_system_message_isolation PASSED [ 10%]
tests/test_prompts.py::test_delimiter_breakout_sanitization PASSED [ 15%]
tests/test_prompts.py::test_empty_input_rejected PASSED [ 21%]
tests/test_prompts.py::test_excessive_length_rejected PASSED [ 26%]
tests/test_schema.py::test_valid_classified_payload PASSED [ 31%]
tests/test_schema.py::test_valid_out_of_scope_payload PASSED [ 36%]
tests/test_schema.py::test_invalid_severity_enum PASSED [ 42%]
tests/test_schema.py::test_invalid_component_enum PASSED [ 47%]
tests/test_schema.py::test_extra_fields_forbidden PASSED [ 52%]
tests/test_schema.py::test_missing_required_field PASSED [ 57%]
tests/test_schema.py::test_cross_field_invariant_p0_requires_urgent PASSED [ 63%]
tests/test_schema.py::test_classified_requires_severity_and_component PASSED [ 68%]
tests/test_tool_loop.py::test_4_stage_agent_tool_loop PASSED [ 73%]
tests/test_tools.py::test_valid_components PASSED [ 78%]
tests/test_tools.py::test_invalid_component_handled_gracefully PASSED [ 84%]
tests/test_tools.py::test_unknown_tool_rejected PASSED [ 89%]
tests/test_tools.py::test_malformed_json_arguments PASSED [ 94%]
tests/test_tool_loop.py::test_4_stage_agent_tool_loop PASSED [100%]

===== 19 passed in 0.89s =====
```

Hình 7: Ảnh chụp kết quả kiểm thử tự động pytest 19/19 test cases pass 100% offline (S08).

```
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Duy\SE373\BTVN02
collected 19 items

tests/test_prompts.py::test_issue_inside_delimiters PASSED [ 5%]
tests/test_prompts.py::test_system_message_isolation PASSED [ 10%]
tests/test_prompts.py::test_delimiter_breakout_sanitization PASSED [ 15%]
tests/test_prompts.py::test_empty_input_rejected PASSED [ 21%]
tests/test_prompts.py::test_excessive_length_rejected PASSED [ 26%]
tests/test_schema.py::test_valid_classified_payload PASSED [ 31%]
tests/test_schema.py::test_valid_out_of_scope_payload PASSED [ 36%]
tests/test_schema.py::test_invalid_severity_enum PASSED [ 42%]
tests/test_schema.py::test_invalid_component_enum PASSED [ 47%]
tests/test_schema.py::test_extra_fields_forbidden PASSED [ 52%]
tests/test_schema.py::test_missing_required_field PASSED [ 57%]
tests/test_schema.py::test_cross_field_invariant_p0_requires_urgent PASSED [ 63%]
tests/test_schema.py::test_classified_requires_severity_and_component PASSED [ 68%]
tests/test_tool_loop.py::test_4_stage_agent_tool_loop PASSED [ 73%]
tests/test_tools.py::test_valid_components PASSED [ 78%]
tests/test_tools.py::test_invalid_component_handled_gracefully PASSED [ 84%]
tests/test_tools.py::test_unknown_tool_rejected PASSED [ 89%]
tests/test_tools.py::test_malformed_json_arguments PASSED [ 94%]
tests/test_tool_loop.py::test_4_stage_agent_tool_loop PASSED [100%]
```

```
===== 19 passed in 0.83s =====
```